01 Einführung

Einführung in die Programmierung

> **Vorlesung 06.10.2026 – Programmieren im Softwareentwicklungsprozess**
> Quelle: 01_Einführung.pdf

Softwareentwicklungsprozess

Vogelperspektive

Softwareentwicklung = Prozess zur Entwicklung eines geeigneten Softwaresystems zur Lösung eines Problems.

`Problem` → (Anforderungen) → `Softwaresystem` → `Lösung`

> **Beispiel aus den Folien**
> Problem: „Wir brauchen ein Softwaresystem, das den Bahnkunden hilft Tickets zu kaufen."
> Lösung: Navigator-App zur Suche nach Verbindungen und zum Kauf von Tickets.

Phasen (Wasserfallmodell)

Das **Wasserfallmodell** ist das älteste und einfachste Vorgehensmodell der Softwareentwicklung.

```
flowchart TD
    A[Anforderungsanalyse] --> B[Entwurf] --> C[Implementierung] --> D[Test] --> E[Auslieferung]
```

> **Begriff**
> **Implementierung** = Programmierung = Realisierung = Umsetzung der Lösung in Code.

Klassen von Programmiersprachen

Es gibt verschiedene Klassen von Programmiersprachen (Programmier-Paradigmen)

- Imperative/prozedurale Programmiersprachen (z.B. Fortran, Cobol, C, Pascal)
  - In der Regel ältere Sprachen (60er bis 80er Jahre), die heute eher selten eingesetzt werden.
  - Aufteilung von komplexen Algorithmen in überschaubare Einheiten (Prozeduren) mit klar definierter Schnittstelle.
  - Die meisten modernen Programmiersprachen unterstützen immer auch prozedurale Aspekte.
- Objektorientierte Programmiersprachen (z.B. Java, C++, C#)
  - Weiterentwicklung der prozeduralen Sprachen in den 90er Jahren.
  - Zentrales Konzept ist die Modularisierung des Codes in Klassen: kapseln Daten (Attributen) und Verhalten (Methoden).
- Funktionale Programmiersprachen (z.B. Lisp, Haskell, F#, Scala)
  - Funktionen stehen im Vordergrund. Daten werden als Parameter übergeben und meist nicht nachträglich verändert.
- Script-Sprachen (z.B. Javascript, Python)
  - Unterstützen in der Regel mehrere der oben genannten Paradigmen.

Der Trend geht heute zu hybriden Sprachen: d.h. Sprachen, die Konzepte verschiedener Paradigmen unterstützen.

7

Syntax, Semantik und Pragmatik von Programmiersprachen

Programmiersprachen sind formale Sprache zur Formulierung von Algorithmen, die von Computern interpretiert und ausgeführt werden können. Sind gekennzeichnet durch:

- Syntax:
  - Gültige Zeichenmenge (Alphabet).
  - Gültige Verknüpfungen der Zeichen des Alphabets zur Bildung von Ausdrücken (Grammatik).
  - Syntax lässt sich gut automatisiert prüfen.
- Semantik:
  - Ordnet Ausdrücken eine konkrete Bedeutung zu.
  - Semantik lässt sich teilweise automatisiert prüfen: statische Prüfung: z.B. korrekte Datentypen durch den Compiler, dynamisches Verhalten zur Laufzeit.
  - Die Semantik wird teilweise erst im Kontext klar (mehrdeutige Ausdrücke).
- Pragmatik:
  - Adäquate Verwendung der Ausdrücke für bestimmte Probleme.
  - Ziel ist die Lesbarkeit und Wartbarkeit des Codes (wie gut kann ein Mensch den Code verstehen und weiterentwickeln).
  - Wird aus der Erfahrung heraus abgeleitet.
  - Lässt sich eingeschränkt automatisiert prüfen, z.B. durch Code Checker.

8

Beispiel Summe 1 bis N

Phase	Inhalt
Anforderungsanalyse	Programm, das die Summe über alle ganzen Zahlen von 1 bis zu einer positiven Obergrenze N berechnet
Entwurf	Lösungsidee, z.B. die **Gaußsche Summenformel** $\sum_{i=1}^{N} i = \frac{N\,(N+1)}{2}$
Implementierung	**allgemeine** Lösung (für unterschiedliche N) in einer Programmiersprache
Test	mit verschiedenen gültigen **und ungültigen** Eingabewerten für N
Auslieferung	Bereitstellung für alle vorgesehenen Nutzer, z.B. Webanwendung in der Cloud, Download, Installation beim Kunden

Code dazu:

Take-aways

- Ziel der Softwareentwicklung: digitale Lösungen (Softwaresysteme) für konkrete Probleme der Praxis.
- **Programmieren** = Beschreibung eines Lösungsalgorithmus mit einer Programmiersprache.
- Programmieren ist zentraler, **aber nicht einziger** Bestandteil des Entwicklungsprozesses.

Programmiersprachen

Was ist eine Programmiersprache?

- Erlaubt die Formulierung von **Algorithmen** in einer für Menschen verständlichen Sprache.
- **Präzise genug** (eindeutig), um von Maschinen interpretiert zu werden.
- **Verständlich genug**, damit Menschen sie nachvollziehen können.

```
flowchart LR
    M[Mathematischer Ausdruck] -->|ggf. Umformung| P[Formulierung in formaler Sprache]
    P -->|Compiler: automatisierte Übersetzung| B[Maschinencode / Bytecode]
```

Klassen / Paradigmen

Paradigma	Beispiele	Kernidee
Imperativ / prozedural	Fortran, Cobol, C, Pascal	meist ältere Sprachen (60er–80er), heute selten. Komplexe Algorithmen werden in überschaubare Einheiten (**Prozeduren**) mit klar definierter Schnittstelle aufgeteilt. Fast alle modernen Sprachen unterstützen auch prozedurale Aspekte

Paradigma	Beispiele	Kernidee
Objektorientiert	Java, C++, C#	Weiterentwicklung der prozeduralen Sprachen in den 90ern. Zentral: **Klassen** kapseln Daten (**Attribute**) und Verhalten (**Methoden**)
Funktional	Lisp, Haskell, F#, Scala	Funktionen im Vordergrund. Daten werden als Parameter übergeben und meist nicht nachträglich verändert
Script-Sprachen	JavaScript, Python	unterstützen meist mehrere der obigen Paradigmen

Trend
Hin zu **hybriden Sprachen**, die Konzepte mehrerer Paradigmen unterstützen.

Syntax, Semantik, Pragmatik

Programmiersprachen sind **formale Sprachen** zur Formulierung von Algorithmen.

	Bedeutung	Prüfbarkeit
Syntax	gültige Zeichenmenge (**Alphabet**) und gültige Verknüpfungen zu Ausdrücken (**Grammatik**)	gut automatisiert prüfbar (Compiler)
Semantik	ordnet Ausdrücken eine konkrete **Bedeutung** zu; teils erst im Kontext klar (mehrdeutig)	teilweise: **statisch** (z.B. korrekte Datentypen durch den Compiler) und **dynamisch** (Verhalten zur Laufzeit)
Pragmatik	**adäquate Verwendung** der Ausdrücke für bestimmte Probleme. Ziel: Lesbarkeit und Wartbarkeit. Aus Erfahrung abgeleitet	eingeschränkt, z.B. durch Code Checker

Beispiele

Ebene	Deutsch	Java	Wer merkt es?
Syntax (Zeichen)	„A44Aafährt mit dem Fahrrad“	`int value = @;`	Compiler
Syntax (Grammatik)	„Anna fahren mit dem Fahrrad“	`int value =: 2;`	Compiler
Semantik	„Max dreht am Rad“ (nervös? oder dreht wirklich ein Rad?)	`3 / 0` (Division durch 0 nicht erlaubt)	meist erst zur **Laufzeit**

Ebene	Deutsch	Java	Wer merkt es?
Pragmatik	„Tom und Harry fahren mit einem vierrädrigen Vehikel“	`int yes = 0; int no = 1;` (ungünstige Datenstruktur für Ja/Nein, besser boolean)	niemand: keine echten Fehler, aber Unschönheiten, die zu Fehlern führen können

Merksatz
Syntax (und teilweise Semantik) prüft der **Compiler**, viele semantische Fehler fallen erst zur **Laufzeit** auf.

Java

Struktur objektorientierter Programme

```
Programm
  Klasse(n)
      Membervariablen
      Methoden
```

Element	Beschreibung
Programm	besteht aus **mindestens einer Klasse**. Programmname = Name der **Startklasse**; benennt üblicherweise die Aufgabe bzw. das Problem
Klasse	repräsentiert ein **Konzept** des Problems mit Verhalten und Zuständen. Jede Klasse steht in einer **eigenen Datei mit gleichem Namen**
Membervariablen	Instanzvariablen, Klassenvariablen, Konstanten. Beschreiben Eigenschaften einer Klasse; beliebig viele, auch keine
Methoden	beschreiben das **Verhalten**; berechnen etwas und/oder verändern Werte von Instanzvariablen; beliebig viele, auch keine

Beispiel Gauss

```java
class Gauss {
    // Membervariablen (hier leer)

    // Methoden
    public static void main(String[] args) {

        // Einlesen der Obergrenze von der Kommandozeile
        int boundary = Integer.parseInt(args[0]);

        // Berechnung der Summe nach Gauss
        int sum = boundary * (boundary + 1) / 2;
```

```
        // Alternative Berechnung der Summe
        // int sum = (n * n + n) / 2;

        // Ausgabe des Ergebnisses
        System.out.println(sum);
    }
}
```

- `main` ist der Einstiegspunkt; `args[0]` ist das erste Kommandozeilenargument.
- `Integer.parseInt` wandelt den String in einen `int`.
- Zu Entwurf und Phasen:

Programmierumgebungen

Ablauf: `Gauss.java` (Programm) → **Compiler** → `Gauss.class` (Bytecode) → **Interpreter** → Ausgabe

	Variante 1 (Konsole)	Variante 2 (IDE, Standard im Kurs)
Schreiben	im Java-Editor	in der IDE
Übersetzen	`javac Gauss.java`	automatisch durch den Compiler der IDE
Ausführen	`java Gauss`	in der IDE (z.B. grüner Pfeil)
Ausgabe	Konsole (cmd, PowerShell, Terminal, bash)	Terminal in der IDE

IDE = *Integrated Development Environment*, z.B. IntelliJ, Visual Studio Code, NetBeans, Eclipse.

JDK und JRE

	JRE (Java Runtime Environment)	**JDK** (Java Development Kit)
Enthält	–	ein vollständiges JRE **plus** Entwicklungswerkzeuge (`javac`, `java`, `javadoc`, …)
Aufgaben	Schnittstelle zum Betriebssystem (simuliert dessen Dienste), Ablaufumgebung für den Interpreter (Bytecode), **Speicherverwaltung (Garbage Collector)**, Bibliotheken	Erkennbar am Kürzel „JDK“ im Namen der Download-Datei

> **Achtung**
> Zum Programmieren braucht man **immer beides** → daher immer das **JDK** herunterladen.

Geschichte und Versionen

Jahr	Ereignis
1995	erste Urversion von **Sun Microsystems**
1998	Java 1.2 (J2SE): erste ernstzunehmende Version
2004	Java 1.5 → Java 5: umfassende Überarbeitung und Erweiterung
2010	**Oracle** kauft Sun, entwickelt Java als quelloffene Software weiter
2014	Java 8: funktionale Aspekte
2018	Java 11: erstmals kostenpflichtig im produktiven Einsatz (Entwicklung/Test weiter frei), sehr häufige Versionswechsel

OpenJDK (openjdk.org): offizielle freie Implementierung der Java Plattform (Standard Edition), entstanden als Abspaltung von Java 6.
- aktuelle **LTS**-Version (Long Term Support): OpenJDK 25
- aktuelle Version: OpenJDK 27
- **Praktikum: OpenJDK 25 bis 27**

[01_Einführung.pdf, p.15]

> **Take-away**
> Viele grundlegende Konzepte findet man in anderen Sprachen wieder; der Unterschied liegt vor allem in der **Syntax**.